Université Hassan 1er — FST Settat
Département Physique Appliquée
Année Universitaire : 2026–2027
Module : Cloud Computing, Virtualisation & DevOps
Enseignant : Pr. Abdelfattah Abassi

Série de TP n°5

Docker Compose : premiers pas

(Suite du TP n°4 — Docker : conteneurisation et images)

Durée indicative : 2H30

Consignes générales

Dans le TP n°4, plusieurs conteneurs ont été lancés avec de longues commandes `docker run`. Ici, on apprend à **décrire plusieurs conteneurs dans un seul fichier** (`docker-compose.yml`) et à les démarrer avec **une seule commande**. Les parties s'enchaînent : les réaliser **dans l'ordre**.

Réponses : les questions et les traces demandées sont à compléter dans le fichier Markdown fourni `TP5_Reponses.md` (à renommer `TP5_Reponses_NOM_Prenom.md`). Ne pas modifier les titres du fichier.

Prérequis techniques

- Docker et Docker Compose installés (`docker compose version`)
- Avoir terminé le TP n°4
- Ports libres sur l'hôte : 5000, 8080, 8081
- Une connexion Internet ponctuelle (images `nginx`, `postgres`, `adminer`, `python:3.11-slim`)

Rappel : le format YAML

Un fichier Compose est écrit en YAML : l'**indentation** (2 espaces par niveau, **jamais de tabulation**) fait partie de la syntaxe. Une liste commence par un tiret (`- `). Une erreur d'indentation est la cause d'erreur n°1 dans ce TP.

1 Partie 1 — Mon premier fichier Compose

Objectif

Écrire un fichier Compose minimal et connaître les 4 commandes de base : `up`, `ps`, `logs`, `down`.

1.1 Créer un dossier `tp5-compose` et s'y placer.

1.2 Dans ce dossier, créer le fichier `docker-compose.yml` :

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

1.3 Démarrer le service en arrière-plan :

```
docker compose up -d
```

1.4 Vérifier l'état et tester dans le navigateur (`http://localhost:8080`) :

```
docker compose ps
docker network ls
```

1.5 Afficher les journaux, puis tout arrêter et supprimer :

```
docker compose logs web
docker compose down
```

Vérification attendue

La page d'accueil de Nginx s'affiche sur `http://localhost:8080`. Après `docker compose down`, `docker compose ps` ne liste plus aucun service.

Questions 1.1 et 1.2

Q1.1 — En plus du conteneur, quel autre objet Docker a été créé par `docker compose up` (voir `docker network ls`) ? Quel est son nom ?

Q1.2 — Quelle est la différence entre `docker compose stop` et `docker compose down` ?

2 Partie 2 — Deux services qui communiquent

Objectif

Lancer une base de données PostgreSQL et son outil d'administration web Adminer, et comprendre que les services se joignent **par leur nom**.

2.1 Remplacer le contenu de `docker-compose.yml` par :

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret

  adminer:
```

```
    image: adminer
    ports:
      - "8081:8080"
```

2.2 Démarrer les deux services et vérifier leur état :

```
docker compose up -d
docker compose ps
```

2.3 Ouvrir `http://localhost:8081` et se connecter avec : système *PostgreSQL*, serveur `db`, utilisateur `postgres`, mot de passe `secret`.

2.4 Dans Adminer, repérer la base `postgres` et prendre une capture d'écran.

Vérification attendue

`docker compose ps` liste deux services (`db` et `adminer`) à l'état *running*. La connexion à Adminer réussit.

Questions 2.1 et 2.2

Q2.1 — Dans Adminer, pourquoi écrit-on `db` comme serveur et non `localhost` ?

Q2.2 — Lequel de ces deux services publie un port vers l'hôte (`ports`) ? Pourquoi le service `db` n'en a-t-il pas besoin ici ?

3 Partie 3 — Conserver les données avec un volume

Objectif

Ajouter un volume nommé pour que les données de la base survivent à la suppression des conteneurs.

3.1 Modifier `docker-compose.yml` : ajouter les lignes `volumes` au service `db`, et la déclaration du volume à la fin du fichier :

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - data_db:/var/lib/postgresql/data

  adminer:
    image: adminer
    ports:
      - "8081:8080"

volumes:
  data_db:
```

3.2 Appliquer la modification (Compose ne recrée que ce qui a changé) :

```
docker compose up -d
```

3.3 Créer une table de test dans la base :

```
docker compose exec db psql -U postgres -c "CREATE TABLE etudiants (id
    int, nom text);"
docker compose exec db psql -U postgres -c "INSERT INTO etudiants VALUES
    (1, 'Sara');"
docker compose exec db psql -U postgres -c "SELECT * FROM etudiants;"
```

3.4 Supprimer les conteneurs (**sans** le volume), les recréer, et relire la table :

```
docker compose down
docker compose up -d
docker compose exec db psql -U postgres -c "SELECT * FROM etudiants;"
```

3.5 Vérifier l'existence du volume :

```
docker volume ls
```

Vérification attendue

À l'étape 3.4, la ligne `1 | Sara` est toujours présente après `down` puis `up`. Un volume dont le nom se termine par `data_db` apparaît dans la liste.

Questions 3.1 et 3.2

Q3.1 — Pourquoi les données sont-elles toujours là après `docker compose down` puis `up` ?

Q3.2 — Que se serait-il passé sans la ligne `volumes` dans le service `db` ? (Se référer à la Partie 4 du TP n°4.)

4 Partie 4 — Sortir le mot de passe avec un fichier `.env`

Objectif

Ne plus écrire le mot de passe en clair dans le fichier Compose, en utilisant une variable.

4.1 Créer, à côté de `docker-compose.yml`, un fichier nommé exactement `.env` :

```
POSTGRES_PASSWORD=secret
```

4.2 Dans `docker-compose.yml`, remplacer la ligne du mot de passe du service `db` par :

```
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

4.3 Vérifier que la variable est bien remplacée (sans rien démarrer) :

```
docker compose config
```

4.4 Appliquer et vérifier que tout fonctionne toujours :

```
docker compose up -d
docker compose ps
```

4.5 Créer un fichier `.gitignore` contenant une seule ligne : `.env`

Attention

Garder le **même mot de passe** (`secret`) que dans la Partie 3 : PostgreSQL ne lit le mot de passe qu'à la création du volume. Le changer maintenant empêcherait de se connecter à la base existante.

Vérification attendue

`docker compose config` affiche `POSTGRES_PASSWORD: secret` : Compose a remplacé `${POSTGRES_PASSWORD}` par la valeur du fichier `.env`.

Questions 4.1 et 4.2

Q4.1 — Quel est l'intérêt d'utiliser un fichier `.env` plutôt que d'écrire le mot de passe directement dans `docker-compose.yml` ?

Q4.2 — Pourquoi ne faut-il pas envoyer le fichier `.env` sur un dépôt Git partagé ?

5 Partie 5 — Ajouter sa propre application

Objectif

Faire construire une image par Compose (`build`) et la démarrer avec les autres services.

5.1 Créer un sous-dossier `api` avec trois fichiers (identiques à ceux du TP n°4). Fichier `api/app.py` :

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Bonjour depuis Docker Compose !"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Fichier `api/requirements.txt` :

```
flask==3.0.0
```

Fichier `api/Dockerfile` :

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

5.2 Ajouter le service `api` dans `docker-compose.yml`, sous `services` (au même niveau que `db` et `adminer`) :

```
  api:
    build: ./api
    ports:
      - "5000:5000"
    depends_on:
      - db
```

5.3 Construire l'image et démarrer l'ensemble :

```
docker compose up -d --build
docker compose ps
curl http://localhost:5000
```

5.4 Vérifier que le service `api` voit le service `db` par son nom :

```
docker compose exec api python -c "import socket; print(socket.
    gethostbyname('db'))"
```

5.5 Modifier le message dans `api/app.py`, puis relancer **seulement** :

```
docker compose up -d
curl http://localhost:5000
```

Observer que le message n'a pas changé, puis relancer avec la reconstruction :

```
docker compose up -d --build
curl http://localhost:5000
```

Vérification attendue

`docker compose ps` liste trois services (`db`, `adminer`, `api`) à l'état *running*. La commande de l'étape 5.4 affiche une adresse IP (par exemple `172.18.0.2`). À l'étape 5.5, le nouveau message n'apparaît qu'après `-build`.

Questions 5.1 à 5.3

Q5.1 — Quelle est la différence entre `image: postgres:15` et `build: ./api` ?

Q5.2 — À quoi sert `depends_on: - db` ? Garantit-il que la base est *prête* à re-

cevoir des connexions, ou seulement que son conteneur a été *démarré* avant ?

Q5.3 — Pourquoi `docker compose up -d` seul n'a-t-il pas pris en compte la modification de `app.py` ? Quelle option faut-il ajouter ?

6 Partie 6 — Nettoyage et vérification finale

Objectif

Nettoyer l'environnement en comprenant l'effet de chaque commande.

6.1 Arrêter et supprimer les conteneurs et le réseau, **en conservant** le volume :

```
docker compose down
docker volume ls
```

Attention

La commande de l'étape suivante est **irréversible** : elle supprime aussi le volume et donc les données de la base. Ne l'exécuter qu'après avoir terminé et documenté tout le TP.

6.2 Nettoyage complet, volumes compris :

```
docker compose down -v
```

6.3 Exécuter les commandes de vérification finale et copier leurs sorties dans le fichier de réponses :

```
docker ps -a
docker volume ls
docker network ls
```

Question 6.1

Q6.1 — Quelle option de `docker compose down` supprime les volumes ? Pourquoi doit-on l'utiliser avec précaution pour une base de données réelle ?

À rendre

Le fichier `TP5_Reponses_NOM_Prenom.md` complété, contenant :

- les réponses aux questions Q1.1 à Q6.1 ;
- les traces demandées (sorties de terminal collées dans les blocs de code, capture d'écran d'Adminer) ;
- le `docker-compose.yml` final et le fichier `api/Dockerfile` ;
- une courte conclusion (5 lignes maximum) comparant Docker Compose aux commandes `docker run` du TP n°4.

Ne pas inclure le fichier `.env` réel : seulement son nom de variable.